

An Efficient Algorithm for Mining Top- k High-On-Shelf-Utility Itemsets with Positive/Negative Profits of Local/Global Minimum Count [†]

Ye-In Chang , Po-Chun Chuang , Yu-Hao Liao, Po-Yu Hu and Ting-Wei Chen

Department of Computer Science and Engineering, National Sun Yat-Sen University, Kaohsiung 80424, Taiwan; zhungboqun@gmail.com (P.-C.C.); karta88821@gmail.com (Y.-H.L.); hupy@db.cse.nsysu.edu.tw (P.-Y.H.); chentw@db.cse.nsysu.edu.tw (T.-W.C.)

* Correspondence: changyi@cse.nsysu.edu.tw

[†] Presented at the 2025 IEEE 5th International Conference on Electronic Communications, Internet of Things and Big Data, New Taipei, Taiwan, 25–27 April 2025.

Abstract

High-utility itemset mining (HUIM) utilizes the threshold value to extract HUI from the transactional database. However, it is difficult to define an optimal threshold value, since it depends on the domain knowledge of the application. Therefore, top- k HUIM is used to solve the problem of setting a threshold. A user can define a k value, which represents the number of HUIs. Moreover, there exist itemsets occurring at a specific time interval, which can become HUI. Since the traditional HUIM algorithm does not consider the transaction with the time interval, the HUIM algorithm cannot be used directly. Therefore, high-on-shelf-utility itemset mining (HOUIM) is used to address the above problem in this study. The proportion of the utility value of the item in all of the time intervals with the itemset is used for determining whether the itemset is HOUI or not. In the top- k HOUIM, the KOSHU algorithm is used based on the data structure, ignoring the item with the negative profit in overestimating the utility of the itemset. The KOSHU algorithm needs less processing time. However, the KOSHU algorithm has to scan the database twice and sort the database once. Therefore, we developed an efficient algorithm based on the TIPN table to mine top- k HOUIs. The developed data structures include TIPN and MINC tables, IO Bitmap, and TIUL. In the TIPN table, we recorded positive items, positive utilities, negative items, and negative counts. The MINC table is used for storing the local/global counts of all of the items with negative profits. In the algorithm, we scanned the database only once. The developed algorithm is more efficient than the KOSHU algorithm.

Keywords: data mining; high-on-shelf-utility itemset mining; negative unit profits; static transactional database; top- k high utility itemset mining



Academic Editors: Teen-Hang Meen, Shu-Han Liao and Cheng-Fu Yang

Published: 16 September 2025

Citation: Chang, Y.-I.; Chuang, P.-C.; Liao, Y.-H.; Hu, P.-Y.; Chen, T.-W. An Efficient Algorithm for Mining Top- k High-On-Shelf-Utility Itemsets with Positive/Negative Profits of Local/Global Minimum Count. *Eng. Proc.* **2025**, *108*, 45. <https://doi.org/10.3390/engproc2025108045>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

In the frequent weighted itemsets mining (FWIM) [1], the frequency and the weight of the item are considered. High-utility itemsets mining (HUIM) [2,3] has become popular recently. If the utility value of itemset X is not less than the minimum threshold value, then itemset X is an HUI. However, it is difficult to define an appropriate threshold value. To address the issue, the top- k HUIM [3] has been proposed. The default threshold value is set to 0, and then several threshold values are used, such as remaining item utility (RIU) and remaining utility constraint (RUC).

The traditional HUIM algorithm considers all of the transactions in the database on-shelf. High-on-shelf-utility itemset mining (HOUIM) [4] has been proposed by considering the on-shelf time interval of the transaction to find itemsets with high utility in the specific time interval. All of the transactions are added with another field as a time interval, which indicates that the transaction is on-shelf, as shown in Table 1. The related profit of items a, b, c, and d is 2, 4, 1, and 3, respectively.

Table 1. Database D1 with on-shelf time intervals.

TID	Transaction	Interval
T ₁	(a,1)(b,2)(c,1)	1
T ₂	(c,3)(d,2)	1
T ₃	(b,2)(c,4)(d,3)	2
T ₄	(b,6)(d,2)	2

The total utility of a time interval h is denoted as $TIU(h)$, which is calculated as the sum of the utility of transactions in which the time interval is h . The utility of itemsets in a certain time interval is equal to the summation of the utility of all itemsets in a certain interval. The result of the total utility of time interval 1 is 20 and that of time interval 2 is 51. The relative utility of the given itemset X is used for determining whether the itemset is an HOUI.

Singh et al. proposed the top- k high-utility itemset mining (TKEH) algorithm [3] for mining top- k HUIs. Srikumar proposed the THUI algorithm [5] to mine top- k HUIs. Ashraf et al. proposed the TKN algorithm [6]. Later, HOUIM [7] is considered as the time interval. The redefined transaction-weighted utility (RTWU) [1] is a utility overestimated value that ignores items with negative profits. The KOSHU algorithm [4] is constructed based on the utility list structure to mine top- k HOUIs. The KOSHU algorithm reduces the number of candidates during the mining process. However, the KOSHU algorithm needs to scan the database twice.

To reduce the number of database scans in the static database, we developed an efficient algorithm called TIPN-table-based to extract top- k HOUIs. We applied the local and global concepts to deal with items with a negative profit. We also introduced the IO Bitmap to record the occurrence of all of the items according to different time intervals. We proposed two pruning strategies based on the research results. The developed TIPN-table-based algorithm is more efficient than the KOSHU algorithm.

2. KOSHU Algorithm

The KOSHU algorithm [4] mines top- k HOUIs. The KOSHU algorithm calculates RTWU values for all of the items and sorts them in the descending order. Then, it applies two threshold-increasing strategies. Moreover, it uses a pruning strategy to effectively diminish the processing time. However, the KOSHU algorithm requires iterating the database twice to mine the top- k HOUIs. Moreover, the utility list for each item is constructed by the KOSHU algorithm.

3. TIPN-Table-Based Algorithm

We used an example database to illustrate the developed algorithm. We described variables, four data structures, three pruning strategies, and two strategies for increasing the threshold value.

3.1. Example Database

We used an example database D2 to illustrate the algorithm (Table 2). Each item in the set I has its profit = [a:5, b:−2, c:3, d:−1, e:3, f:4]. Each transaction T_j in database D2 contains a unique identifier TID. Moreover, in each transaction T_j, we use a subset of items I with the related count and a related time interval.

Table 2. Example database D₂ with on-shelf time intervals.

TID	Transaction	Interval
T ₁	(b,3)(d,2)(f,4)	1
T ₂	(b,2)(c,7)(d,4)(e,5)	1
T ₃	(a,6)(c,3)(d,4)	2
T ₄	(a,4)(d,2)(e,2)	2
T ₅	(b,3)(c,8)(d,5)(e,4)	3
T ₆	(b,1)(c,6)(d,3)	3

3.2. Definitions and Notations

P_V al(i) and Q_V al(i) represent the profit and the quantity value of item i, respectively. Moreover, UT (i,T_j) is defined as the utility of item i in Transaction T_j, which is calculated as the product of P_V al(i) and Q_V al(i). UT (X,T_j) represents the utility of itemset X in transaction T_j, which is calculated as the cumulative utility. TotalU (T) represents the total utility of transaction T, which is the sum of the utility of each item in transaction T. The total utility of each transaction in database D₂ is 8, 23, 35, 22, 21, and 13, for transactions T₁, T₂, T₃, T₄, T₅, and T₆, respectively. Each product is on the shelf at different time intervals. We defined the list of whole time intervals (TIList) as in database D₂. TotalTI(h) represents the utility of time interval h. The total utilities of all of the time intervals in database D₂ are [a,59], [b,83], [c,106], [d,142], [e,77], and [f,14]. The utility of itemset X in time interval h is denoted as UTI(X,h). The relative utility of the itemset is used for determining whether the itemset is an HOUI or not. RelativeU (X) represents the relative utility of the itemset, which is computed by the utility of itemset X divided by the total utility of each time interval that contains itemset X in the database.

3.3. Data Structures

In the proposed data structure, we stored the information for each transaction in the TIPN table. Negative items are ignored in the overestimated value of the itemset. We designed the NIMC table to keep the local minimum of each negative item according to different time intervals. Obviously, in mining top-k high-on-shelf-utility itemsets, the threshold is initially set to 0, and it is increased during the mining process. We used the bitmap to store the status of the occurrence of each item according to different time intervals. We used the TIList to achieve the goal. The TIPN table has six columns as shown in Table 3. Each row in TIPN_Table contains a set of transactions at that time interval, and each transaction is classified as positive items and negative items. Furthermore, the utility of each positive item in the transaction is stored. Here, we stored the count of each negative item to construct the NIMC table.

We constructed the data structure of the NIMC table to decrease the overestimated utility. Moreover, we used two counts, global noise correction (GNC) and local noise correction (LNC), to record the minimum count of each negative item (b and d). The global minimum count (GMC) of each negative item was also recorded. For each negative item, we calculated the minimum count of each time interval and stored it in the GMC column. When count = 0, we skipped it. For example, for negative item b, we recorded 2, 0, and 1, for time intervals 1, 2, and 3, respectively, and recorded GMC = 1. IO_Bitmap was used

for storing the occurrence of each item in the database, and it was constructed during the initial pass of the database. For example, for item c in time interval 1, the related bits are 01.

Table 3. TIPN table of database D.

Time Interval	TID	Positive Items	Positive Utility	Negative Items	Negative Count
1	T ₁	f	f:16	b,d	b:3, d:2
	T ₂	c,e	c:21, e:10	b,d	b:2, d:4
2	T ₃	a,c	a:30, c:9	d	d:4
	T ₄	a,e	a:20, e:4	d	d:2
3	T ₅	c,e	c:24, e:8	b,d	b:3, d:5
	T ₆	c	c:18	b,d	b:1, d:3

TIList was used to discover HOUIs efficiently based on the utility list. We defined a list of itemset X as $TIUL(X) = \{(TI, \{(TID, P_Util, N_Util, NGC_Util, R_Util)\})\}$, where TI is the time interval, and P_Util is the positive utility of itemset X in transaction TrID. N_Util is the negative utility of itemset X in transaction TID, and NGC_Util is the negative utility of itemset X that considers GMC in transaction T_{T ID}. $UTGC(I, T_j) = TU(i, T_j)$ if $P_Val(i) > 0$. Moreover, $UTGC(i, T_j) = GC(i) \times P_Val(i)$ if $P_Val(i) < 0$. Furthermore, the remaining utility (R_Util) of itemset X in transaction T_{T ID} is defined as follows.

$$R_{Util(X, T_j)} = \sum_{i \in T_j \wedge i > x \forall x \in X} UTLC(i, T_j) \quad (1)$$

where $UTGC(i, T_j)$ adopts the GMC of each negative item according to the NIMC table. Then, the utility of item i is calculated as $UTLC(b, T_1) = GC(b) \times (-4) = -8$.

The TIList of itemset X stores the positive utility and negative utility of itemset X. Moreover, the list stores the negative utility by considering the GMC of each period. Table 4 shows the TIList of item e. In the mining process, the algorithm constructs a list for each single item in the database. Then, the list is used for extracting HOUIs in the mining procedure. To calculate the utility of the itemset and the overestimated utility of the item, we defined variables as follows: (1) sumP_Util(X) means the sum of positive utilities P_Util(X) of TIUL of X; (2) sumN_Util(X) means the sum of negative utilities N_Util(X) of TIUL of X; (3) sumNLC_Util(X) means the sum of negative utilities with local minimum count NNGC_Util of TIUL of X; (4) sumR_Util(X) means the sum of remaining utilities R_Util of TIUL of X; and (5) sumN_Util(X) means the accumulation of positive utilities and negative utilities of TIUL of X.

Table 4. TIList of item e.

Itemset	Time Interval	TID	P_Util	N_Util	NLC_Util	R_Util
e	1	2	10	0	0	15
	2	4	4	0	0	−2
	3	5	8	0	0	19

3.4. Pruning Strategies

We introduced three pruning strategies. In the first pruning strategy called TWUGC, which is motivated by the RTWU [8], we define the TWUGC as follows.

$$TWUGC(X) = \sum_{X \in T_j \wedge T_j \in D} TotalU_GC(T_j) \quad (2)$$

$$TotalU_GC(T_j) = \sum_{i \in T_j \wedge T_j \in D} UTGC(i, T_j). \quad (3)$$

$$UTGC(i, T_j) = \begin{cases} TU(i, T_j), & \text{if } P_Val(i) > 0. \\ GC(i) \times P_Val(i), & \text{if } P_Val(i) < 0. \end{cases} \quad (4)$$

The TWUGC of itemset X is the sum of $TotalU_GC(T_j)$ in transactions where itemset X appears. $TotalU_GC(T_j)$ is calculated as the sum of utilities of items in transactions T_j . If the item is negative, the utility of the item is calculated as the product of GMC of the item and the profit of the item. For the second pruning strategy, the RLC pruning strategy prunes hopeless candidates. We utilized the utility list to calculate the overestimated utility value of the relative utility of the itemset. For the third pruning strategy, the TIO pruning strategy prunes the subtrees of the set-enumeration tree which do not appear in the database during the mining process. We introduced the IO Bitmap, which recorded the occurrence statuses of all of the items.

3.5. Threshold Increased Strategies

We introduced two strategies with increased thresholds. The RPRU_Size1 strategy was used to calculate the relative utility for all of the positive items in database D and insert those positive items into the RPRU_Size1_List. Then, the RPRU_Size1 strategy sorts the RPRU_Size1 list according to the descending order of relative utilities, and it increases the threshold value to the k -highest relative utility in the RPRU_Size1 list. The RRU_Size2 strategy was used to calculate the relative utilities for all of the items in database D and insert them into RRU_Size2 list. The RRU_Size2 strategy sorts the RRU_All_List according to the descending order of relative utilities, and it increases the threshold value to the k -highest relative utility in the RRU_All_List.

3.6. Mining Process

The mining process of the developed algorithm is as follows.

3.6.1. Preprocessing Step

In this step, we constructed TIList, TotalTI table, TIPN table, IO table, and NIMC table. In addition, LMC and GCM were added. After scanning the database once, the algorithm performed the RPRU-Size1 strategy to increase the value of ThreVal. To obtain the real relative utility of positive item a , we calculated the utility of positive item a by using the TIPN table. The results showed that $UD(a) = 115$ $TI_Occu_List(a) = \{1,2,3\}$, and $TI_Occu_Total(a) = 133$. The real relative utility of item a is calculated as $UD(a)/TI_Occu_Total(a) = 0.86$. The result of RPRU_Size1 list, which stores real relative utilities of positive items, is [Each positive item, RPRU] = $[[a,0.86], [c,0.54], [e,0.17], [f,0.09]]$. If the size of RPRU_Size1 list was not less than k , we increased the threshold value to the k -highest value in RPRU_Size1_List. On the other hand, TK_List stored the k -highest HOUIs by the descending order of the relative utility. Therefore, TK_List was updated by RPRU_Size1_List.

Then, we calculated the TWUGC value of all of the items in the database to overestimate the relative utility. We defined a total order (TWUGC_Order) for finding the top- k HOUIs efficiently. TWUGC_Order has three ordering rules. First, positive items are sorted

by the descending order of TWUGC. Second, negative items are sorted by the descending order of TWUGC. Third, negative items are sorted in positive items. TWUGC_Order of database D2 is [f,a,e,c,b,d]. Table 5 shows the result of the sorted TIPN table.

Table 5. Sorting TIPN table in TWUGC.

Time Interval	TID	Positive Items	Positive Utility	Negative Items	Negative Count
1	T ₁	f	f:16	b,d	b:3, d:2
	T ₂	e,c	c:21, e:10	b,d	b:2, d:4
2	T ₃	a,c	a:30, c:9	d	d:4
	T ₄	a,e	a:20, e:4	d	d:2
3	T ₅	e,c	c:24, e:8	b,d	b:3, d:5
	T ₆	c	c:18	b,d	b:1, d:3

After the TIPN table was sorted by the TWUGC_Order, the developed algorithm created a TIList for all of the single items in the database. The TIList of each item was used for discovering itemsets with a large size by using the itemset expansion method.

To increase the minimum threshold value for finding top-k HOUIs, the algorithm applied another threshold-increased strategy called the RRU-Size2 strategy. In the RRU-Size2 strategy, the real relative utility is calculated for each size 2 itemset and stored in the RRU_Size2 list. Moreover, the algorithm uses the Time Interval_Utility list and IO_Table to calculate the real relative utility of the size 2 itemset efficiently. For obtaining the real relative utility of itemset $x \cup y$, we need to calculate the utility of itemset $x \cup y$ and the total utility of each time interval which contains the occurrence of itemset $x \cup y$. The real relative utility of $x \cup y$ can be calculated as $\text{Relative}(x \cup y) = \text{UD}(x \cup y) / \text{TI_Occu_Total}(x \cup y)$. The TK list is updated by the RRU_ALL list.

3.6.2. Mining Step

In the mining step, the algorithm applies the pattern growth method to discover top-k HOUIs. At the beginning of the mining process, the algorithm traverses through all of the TIULs of items from the root node of the set enumeration tree, i.e., empty itemset. In each iteration, the original itemset P is appended with the current item x to obtain a new itemset $\text{NewP} = P \cup \{x\}$. Then, the algorithm checks whether the new itemset NewP is an HOUI or not. If the relative utility of the itemset is not less than the threshold value ThreVal , the itemset is a HOUI. If the relative utility of the new itemset NewP is greater than the k -highest relative utility in the TK list, the algorithm removes the k -highest itemset and inserts the new itemset NewP into the TK list. Moreover, the threshold value ThreVal is updated as the k -highest relative utility in the TK list. To calculate the relative utility of new itemset NewP , we calculated the utility of new itemset NewP in the database and the total utility of each time interval that contains the occurrence of the new itemset in the database TI_Occu_Total . For calculating the utility of new itemset NewP , the algorithm uses TIUL of itemset NewP . The sum of positive utilities and negative utilities of itemset NewP contains $\text{sumPN_Util}(\text{NewP})$, which is the accumulation of positive utilities $\text{sumP_Util}(\text{NewP})$ and the accumulation of negative utilities $\text{sumN_Util}(\text{NewP})$.

If $\text{sumPN_Util}(\text{NewP})$ is less than 0, the algorithm skips it directly. For calculating TI_Occu_Total , the algorithm performs the AND operation of all of the time intervals of itemset NewP in IO_Bitmap . If the related utility of the new itemset NewP is not less than the threshold value ThreVal , the new itemset NewP is an HOUI. The objective is to discover the top- k HOUIs within the database. Moreover, the algorithm creates a list called TK list

to store the top-k HOUIs during the mining process. The TK list is a list that dynamically sorts all of the HOUIs in the TK list according to the descending order of relative utilities. If the size of the TK list is less than the user-defined parameter k , the algorithm inserts the new itemset NewP into the TK list directly. If the size of the TK list is equal to the user-defined parameter k , the algorithm checks whether the relative utility of new itemset NewP is greater than the k -highest relative utility in the TK list or not. If the result is true, the new itemset NewP is inserted into the TK list. After the above checking, whether the new itemset NewP is HOUI or not, the algorithm applies the TWUGC pruning strategy to prune unpromising itemsets.

4. Performance Evaluation

The TIPN-table-based algorithm and the KOSHU algorithm were evaluated for their performance [4]. For evaluation, we utilized two databases: real and synthetic databases.

4.1. Performance Model

The real database is the sparse database downloaded from the SPMF library [8]. The retail database has a density of lower than 1% as a sparse database. The value of the time interval is 5, which is equal to the consideration of the KOSHU algorithm [4]. We set k to the range between 50 and 150. The real database has 88162 transactions (containing 16470 items) with density = 0.06. For the synthetic database, we utilized four parameters T , I , MI , and NP to experiment with the TIPN-table-based algorithm and KOSHU algorithm, where T represents the total amount of transactions in the database, and I represents the total number of distinct items in the database. MI represents the maximum number of distinct items of a single transaction, and NP represents the percentage of counts of items with negative profits. For example, $T_{10,000_I4000_MI10_NP_{80}}$ is a synthetic database with 10,000 transactions, 4000 distinct items, up to 10 distinct items in a single transaction, and 80% of items with negative profits. These synthetic databases are obtained from the IBM Almaden Quest research group [9,10].

4.2. Experiment Results

We compared the performance of the TIPN-table-based algorithm and the KOSHU algorithm [4]. The KOSHU algorithm produced the EMPRS data structure during the preprocessing step, which was time-consuming. Moreover, the number of candidates for the TIPN-table-based algorithm is smaller than that of the KOSHU algorithm. There are two reasons why the algorithm could generate fewer candidates than the KOSHU algorithm. First, the TWUGC pruning strategy considered the GMC of all of the items with negative profits. Second, the RLC pruning strategy utilizes the LMC of all of the items with negative profits at each time interval. Therefore, the remaining utility of the RLC pruning strategy is tighter than that of the KOSHU algorithm. Moreover, the number of candidates pruned by the algorithm was more than that of the KOSHU algorithm during the mining process.

Figure 1 shows the comparisons of the performance between these two algorithms by using the real retail database. As the value of k increased, the performance measures, including the processing time and the number of candidates, of our proposed algorithm were better than those of the KOSHU algorithm. Moreover, the reasons for such results are the same as those reasons described before. For the synthetic databases $T_{10,000_I100_MI10_NP_{80}}$ (the dense database), Figure 1c,d show the comparisons of the two concerned algorithms. The result are similar to the comparison between the two algorithms.

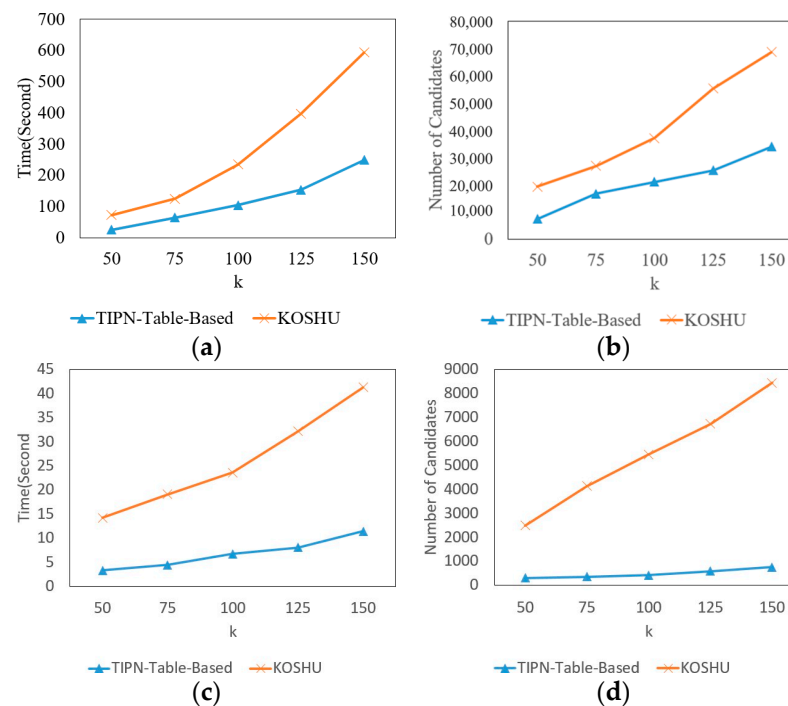


Figure 1. A comparison under the change of k . (a) The processing time by using the real retail database. (b) The total number of candidates by using the real retail database. (c) The processing time by using the synthetic database T10,000_I100_MI10_NP80by. (d) The total number of candidate using the synthetic database.

5. Conclusions

In this study, we developed the TIPN-table-based algorithm to mine top- k high on-shelf utility itemsets efficiently. The TIPN-table-based algorithm only scans the database once and sorts the database once. Moreover, we proposed global and local concepts to make the tight upper bound. We utilized a bit map strategy to decrease the processing time. The experiment results showed that the TIPN-Table-Based algorithm performed better than the KOSHU algorithm.

Author Contributions: Conceptualization, Y.-I.C., Y.-H.L. and T.-W.C.; Methodology, T.-W.C. and P.-Y.H.; Writing—original draft preparation, Y.-H.L. and P.-C.C.; Writing—review and editing, T.-W.C. and Y.-I.C. All authors have read and agreed to the published version of the manuscript.

Funding: This research was supported in part by the Office of Research and Development, National Sun Yat-sen University under Grant No. 14DS02.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Singh, K.; Kumar, A.; Singh, S.S.; Shakya, H.K.; Biswas, B. EHNL: An efficient algorithm for mining high utility itemsets with negative utility value and length constraints. *Inf. Sci.* **2019**, *484*, 44–70. [\[CrossRef\]](#)
2. Lee, J.; Yun, U.; Lee, G.; Yoon, E. Efficient incremental high utility pattern mining based on pre-large concept. *Eng. Appl. Artif. Intell.* **2018**, *72*, 111–123. [\[CrossRef\]](#)
3. Singh, K.; Singh, S.S.; Kumar, A.; Biswas, B. TKEH: An efficient algorithm for mining top- k high utility itemsets. *Appl. Intell.* **2018**, *49*, 1078–1097. [\[CrossRef\]](#)
4. Dam, T.-L.; Li, K.; Fournier-Viger, P.; Duong, Q.-H. An efficient algorithm for mining top- k on-shelf high utility itemsets. *Knowl. Inf. Syst.* **2017**, *52*, 621–655. [\[CrossRef\]](#)

5. Krishnamoorthy, S. Mining top-k high utility itemsets with effective threshold raising strategies. *Expert Syst. Appl.* **2019**, *117*, 148–165. [[CrossRef](#)]
6. Chen, J.; Guo, X.; Gan, W.; Chen, C.-M.; Ding, W.; Chen, G. On-shelf utility mining from transaction database. *Eng. Appl. Artif. Intell.* **2022**, *107*, 1–12. [[CrossRef](#)]
7. Ashraf, M.; Abdelkader, T.; Rady, S.; Gharib, T.F. TKN: An efficient approach for discovering top-k high utility itemsets with positive or negative profits. *Inf. Sci.* **2022**, *587*, 654–678. [[CrossRef](#)]
8. Fournier-Viger, P.; Lin, C.W.; Gomariz, A.; Gueniche, T.; Soltani, Z.D.A.; Lam, H.T. The SPMF open-source data mining library version 2. In Proceedings of the 19th European Conference on Principles of Data Mining and Knowledge Discovery (PKDD 2016), Part III, Riva del Garda, Italy, 19–23 September 2016; Springer: Riva del Garda, Italy, 2016; pp. 36–40.
9. IBM. IBM Quest Synthetic Data Generation Code. Available online: <https://www.ibm.com/docs/en/watsonx/w-and-w/2.1.0?topic=data-generating-synthetic> (accessed on 5 July 2025).
10. Vo, B.; Bui, H.; Vo, T.; Le, T. Mining top-rank-k frequent weighted itemsets using WN-list structures and an early pruning strategy. *Knowl.-Based Syst.* **2020**, *201–202*, 1–12. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.